VOXLINE

BUILD HANDBOOK · VERSION 2.0 · AUGUST 2026

Ten working days to *live*

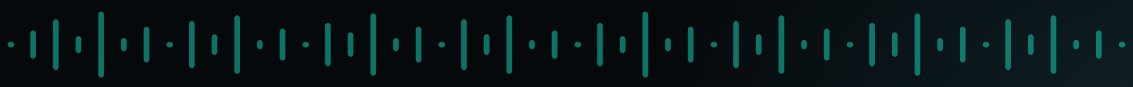
The purpose, the architecture, every screen with a picture of exactly how it should look, the design system, and a day-by-day plan that puts two travel agencies on Voxline by Friday 4 September.

PREPARED FOR
Adnan
Intern engineer, full stack

PREPARED BY
Khush Mutha
Oltaflock AI LLP

WINDOW
24 Aug to 4 Sep
10 working days, hard stop

SCOPE
Portal + billing
Phase 1 plus four Phase 2 items



START HERE

How to use this book

Read it end to end on the morning of day one. It takes about forty minutes. Part 5 shows you a picture of every screen you have to build, and part 7 tells you which day each one is due.

What this book decides

The purpose, the architecture, the data model, how every screen looks and behaves, the visual system, the order of work, and the quality bar. If it is written here, it is decided.

What it leaves to you

How you structure components, name files, write tests and solve each day's work. You are the engineer. Nobody will micromanage the inside of a pull request.

The two files you work from

FILE	WHAT IT IS FOR
`Voxline-UI-Prototype.html`	**The design source of truth.** One self-contained file. Open it in a browser, click every screen in both light and dark, use the Site / Login / Portal switcher at the bottom. Everything in part 5 of this book is a screenshot of it. If this book and the prototype ever disagree on how something looks, the prototype wins.
`Voxline-Spec.md`	**The technical contract.** Data model, exact field names, rules, acceptance criteria. If this book and the spec disagree on data or security, the spec wins.

Contents

The one thing to remember

Two agencies must never see each other's data, and the portal must look like software worth $499 a month. Everything else in this book is negotiable under time pressure. Those two are not.

PART 01 · THE PRODUCT

What Voxline actually is

Voxline is a phone line for travel agencies that always gets answered. An AI voice agent picks up, asks the questions a consultant would ask, and files a structured trip brief in the agency's pipeline before the caller has hung up. We sell that as a monthly subscription.

The customer

A travel agency with three to thirty consultants. The phone is their main source of new business and they miss most of it after 6pm, on weekends and in peak season.

The pain

Every unanswered call is a trip somebody else books. The average booking they lose is worth about $4,100, and they have no idea how many they are losing.

The promise

Nothing rings out. Every caller is qualified, every inquiry lands in their pipeline within a minute, and they can see exactly what the line is producing.

What you are building, precisely

The voice agent itself runs on Retell AI. We do not build voice, telephony or speech. **You are building the platform around it**, which is three things in one Next.js codebase:

`your main job`

The client portal

Where each agency logs in and sees its own calls, transcripts, recordings, trip pipeline, usage and billing. Route `/app`.

`minimal, internal`

The admin console

Where Khush onboards a new agency, attaches its Retell agent and phone number, sets its plan. Route `/admin`.

`day 9, thin`

The marketing site

Public pages that sell the product and link into the portal. You deploy the prototype's page as is. Route `/`.

How the money works

This matters because half the portal exists to show it honestly.

PLAN	PER MONTH	INCLUDED MINUTES	OVERAGE	WHO BUYS IT
Starter	$199	800	$0.18 / min	Single-location agency trying it out
Growth	$499	2,500	$0.14 / min	Most agencies. Our default sale
Scale	Custom	6,000+	$0.11 / min	Groups running several brands

When an agency runs out of minutes, the agent keeps answering

We bill the overage on the next invoice. We never stop picking up the phone, because a dropped caller costs the client more than the overage does. The portal has to make usage visible every day so the invoice is never a surprise.

Who uses it, and what Voxline refuses to do

Three roles, and only three

ROLE	WHO THEY ARE	WHAT THEY CAN REACH
Platform admin `in scope`	Khush and the Oltaflock team	The admin console. Every tenant, agent configuration, plans, the change-request queue, and read-only "view as tenant" with a visible banner
Agency owner `in scope`	The person who signed the contract	The full portal for their own agency or agencies, including billing
Agency member `cut, see p.24`	A consultant on the team	Would be the portal without billing management. Not in these two weeks. Do not design anything that makes it impossible later

A tenant is one agency

Everything in the database belongs to a tenant. One person can belong to two tenants, which is why the portal has an agency switcher in the sidebar: an agency group might run two brands from one login. That is the only reason it exists, so do not turn it into anything cleverer.

USER

Sofia Marchetti

One login, one password, belongs to two tenants through the memberships table

→

TENANT A

Blue Harbor Travel

Growth plan. Its own calls, leads, invoices, agent configuration

+

TENANT B

Wanderlux Journeys

Scale plan. Completely separate rows. Switching is instant and total

What Voxline is not, and will not become in v1

Not a CRM

It will push qualified leads into the agency's existing CRM later. It never tries to replace it. The trip pipeline is a working view of inbound inquiries, not a sales system.

Not a telephony product

Retell owns the phone numbers, the call handling and the speech. We consume their webhooks. If you find yourself reading about SIP trunks, stop, you are off the map.

Not a booking or payment engine

The agent qualifies, humans sell. Anything involving money or a confirmed reservation is handed to a consultant. This is a product decision, not a technical limit.

Not client-editable

Clients cannot edit their voice agent's configuration. A bad edit breaks a live phone line. They send a change request, we make the change. Keep it concierge and say so in the UI.

Say this back to yourself before you start

"Voxline answers a travel agency's phone when a human cannot, captures a structured trip brief, and shows the agency what that line produced. I am building the portal they log into and the pipes that fill it."

PART 02 · THE CORE FLOW

How a call becomes revenue

This is the one path the entire product exists to serve. Steps three and four are built by Khush or Vineet in parallel with your first days, because they are the riskiest part of the system and the schedule cannot absorb them going wrong. You consume what they produce.

1. **A traveller calls the agency at 9:40pm**

   The agency forwarded its existing number to the number we provisioned in Retell, either permanently or only after hours and when the line is busy. Nothing about their phone system changed.

2. **The Retell agent answers and qualifies**

   It greets the caller by the agency's name, in the agency's tone, and works through the qualification questions we configured: destination, travel dates, party size, budget, occasion. When the call ends, Retell runs its post-call analysis and extracts those fields as structured data.

3. **Retell fires a webhook at our API** `built for you, ticket S-2`

   A POST lands on `/api/webhooks/retell` carrying the call ID, duration, recording URL, transcript and analysis. The handler verifies the signature, finds which tenant owns that Retell agent ID, and upserts a row into `calls`. Upsert, never insert: Retell retries, and a retry must not create a second call.

4. **A qualified call becomes a lead, and minutes are counted** `ticket S-2`

   If the outcome is `inquiry_captured` or `quote_requested`, a `leads` row is created in the New inquiry stage, linked back to the call. The call's minutes are added to the tenant's current usage period and reported to Stripe.

5. **It appears in the agency's portal within 60 seconds** `your work starts here`

   The consultant opens the portal next morning and sees a new card in the pipeline: `Amelie Fournier · Greece · Late Sept · 2 pax · ~$6,000`. One click gives them the recording and the full transcript.

6. **The consultant calls back and sells the trip**

   That is the whole product. Everything else, the charts, the billing tab, the outcome breakdown, exists so the agency can see this happening often enough to keep paying for it.

The three ways this flow breaks in production

1. The webhook is not idempotent, so a Retell retry creates duplicate calls and double-counts minutes. Upsert on `retell_call_id`, which is unique.

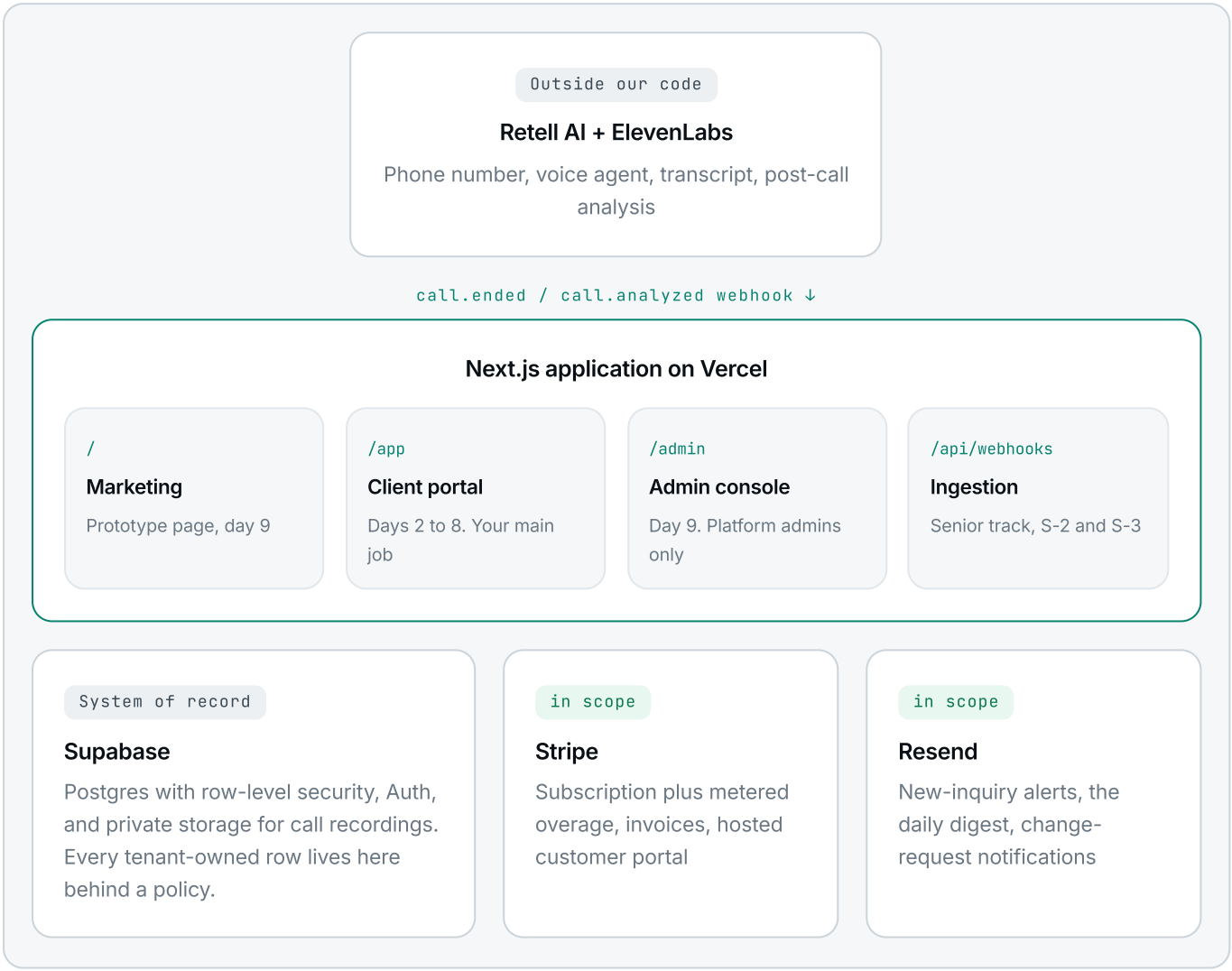
2. A webhook fails silently and a call is lost forever. Every webhook is logged with its ID *before* processing, and a nightly job re-pulls recent calls from the Retell API.

3. The tenant lookup is wrong and agency A's call lands in agency B's portal. The tenant is resolved from the Retell agent ID only, never from anything in the caller's payload.

PART 03 · ARCHITECTURE AND STACK

The shape of the system

One Next.js application on Vercel, one Postgres database on Supabase, two webhook endpoints, and two outside services that do the hard parts. That is the entire system. Resist adding to it.



Why each piece is there

LAYER	WHAT WE USE	WHY THIS ONE
Web app	`Next.js 15, App Router, TypeScript strict`	Marketing site, portal, admin and API routes in one codebase and one deploy
Styling	`Tailwind v4 with our tokens`	The prototype's CSS variables map one to one onto a Tailwind theme. Map once, never write a raw hex
Data and auth	`Supabase Postgres and Auth`	Row-level security is how we enforce tenant isolation in the database instead of trusting our own UI code
Voice	`Retell AI, ElevenLabs`	Numbers, call orchestration, transcripts, structured post-call analysis. Building this ourselves would be a company
Payments, email	`Stripe Billing, Resend`	Base subscription plus metered usage is exactly our pricing. Resend is three lines to send a templated email
Everything else	`Hand-rolled SVG charts, Vercel, Sentry`	No charting library for forty lines of SVG. A preview deploy per pull request, and error triage from day one

Repository and environment

How the code is laid out

```
// this is what the day-zero kit already contains. Keep routes and data access separate.
voxline/
├─ app/
│  ├─ (marketing)/          public pages
│  ├─ (portal)/app/         the client portal, auth required
│  │   ├─ overview/ calls/ pipeline/ billing/ agent/
│  ├─ admin/                platform admins only
│  └─ api/
│      ├─ webhooks/retell/route.ts   call ingestion, idempotent, ticket S-2
│      ├─ webhooks/stripe/route.ts   billing events, ticket S-3
│      └─ cron/digest/route.ts       daily digest email, day 9
├─ components/
│  ├─ ui/                   Button, Badge, Card, Input, Modal, Toast
│  ├─ brand/                Logo, WaveRule, WaveLoader, Spotlight
│  └─ portal/               KpiCard, CallRow, KanbanColumn, TenantSwitcher
├─ lib/
│  ├─ supabase/             server client, browser client, generated types
│  ├─ tenant.ts             resolve + guard the active tenant
│  └─ retell.ts             signature verification, payload types
├─ supabase/migrations/     every schema change, in order
├─ supabase/seed.sql        the two demo tenants
└─ e2e/                     Playwright: login, call, stage move, isolation
```

Environment variables

Given to you on day one. They live in `.env.local`, which is gitignored and stays that way.

VARIABLE	SIDE	NOTES
NEXT_PUBLIC_SUPABASE_URL	client	Safe to ship in the bundle
NEXT_PUBLIC_SUPABASE_ANON_KEY	client	Safe. RLS is what protects the data, not this key
SUPABASE_SERVICE_ROLE_KEY	server only	Bypasses every policy. Admin routes and webhook handlers only. Never prefix it with NEXT_PUBLIC
RETELL_API_KEY RETELL_WEBHOOK_SECRET	server only	Reconciliation job, and signature verification on every incoming webhook
STRIPE_SECRET_KEY STRIPE_WEBHOOK_SECRET	server only	Test mode until the pilot goes live. Same signature rule as Retell
RESEND_API_KEY	server only	New-inquiry alerts, daily digest, change requests
CRON_SECRET	server only	Guards the digest and reconciliation cron routes
SENTRY_DSN	both	Already wired in the kit. Do not remove it to quiet the console

The commands you will live in

Every day

```
pnpm dev
pnpm typecheck
pnpm lint
pnpm test
```

Database

```
supabase db diff -f name_of_change
supabase db push
supabase db reset  rebuild + seed
pnpm e2e  Playwright suite
```

Your local environment never points at production

There is a dev Supabase project and a production one. Confusing them is how test data reaches a client's portal and how a reset destroys real recordings. Check the URL in `.env.local` first.

PART 04 · DATA AND ISOLATION

The tables, and what they hold

Ten tables, all created for you on day one in ticket S-1. Every one that belongs to a client carries `tenant_id`. These field names are the contract, and the prototype's fixture data already matches them.

tenant-owned

tenants

id · name · slug · initials · plan_id
status(active|paused|churned)
branding jsonb · created_at

global

profiles, memberships

profiles: id → auth.users · display_name
avatar_initials · theme_pref
memberships: user_id · tenant_id · role(owner|member)

global

plans

id · name(starter|growth|scale)
monthly_price_cents · included_minutes
overage_cents_per_min · stripe_price_ids jsonb

tenant-owned

voice_agents

tenant_id · retell_agent_id · name · phone_number
voice_desc · languages[] · business_hours jsonb
after_hours_behavior · escalation_number
qualification_questions[] · status(live|paused)
crm_connection jsonb · recording_retention_months

tenant-owned

calls the hot table

tenant_id · voice_agent_id
retell_call_id UNIQUE · caller_name · caller_phone
started_at · duration_seconds
outcome(inquiry_captured|quote_requested|
voicemail|not_a_fit)
recording_path · transcript jsonb · analysis jsonb

tenant-owned

leads

tenant_id · call_id nullable · name · summary
stage(new_inquiry|quoted|booked|traveling)
details jsonb · tags[] · assigned_to
position int · created_at · updated_at

tenant-owned

usage_periods, invoices

usage_periods: tenant_id · period_start · period_end
minutes_used · stripe_reported_at
invoices: tenant_id · stripe_invoice_id · number
period_label · minutes · amount_cents · status · pdf_url

tenant-owned

change_requests, audit_log

change_requests: tenant_id · user_id · message
status(open|done) · created_at
audit_log: tenant_id · actor_user_id · action
payload jsonb · created_at

Indexes, in the first migration

```
create unique index on calls (retell_call_id);          idempotency depends on this
create index on calls (tenant_id, started_at desc);   every list query
create index on leads (tenant_id, stage, position);   the kanban
create index on memberships (user_id);                tenant resolution on every request
```

The seed data is your fixture data

`supabase/seed.sql` reproduces the two demo tenants straight from the prototype: Blue Harbor Travel and Wanderlux Journeys, with their calls, leads, invoices and agent configuration. Every screenshot in part 5 is that data. Build against it and your screens will match the pictures.

The rule that must never break

Blue Harbor must never see a single row belonging to Wanderlux. Not through the UI, not through the API, not through a mistyped query. This is enforced in Postgres, not in your React code, because your React code will eventually have a bug.

wrong

Filtering in application code

```
// one forgotten .eq() and the data leaks
supabase.from('calls')
  .select('*')
  .eq('tenant_id', activeTenant)
```

Works until someone writes a query without the filter, or an API route trusts a tenant ID that came from the browser.

right

A policy in the database

```
alter table calls enable row level security;

create policy tenant_read on calls
for select using (
  tenant_id in (
    select tenant_id from memberships
    where user_id = auth.uid()
  )
);
```

Now an unfiltered select returns only that user's tenants. The database refuses, not the UI.

The test that guards it, running from day one

```
// e2e/isolation.spec.ts, runs in CI on every pull request
test('a user cannot read another tenant rows', async () => {
  const sofia = await signInAs('sofia@blueharbortravel.com');   tenant A only
  const { data } = await sofia.from('calls').select('*');      no filter at all

  expect(data.every(c => c.tenant_id === TENANT_A)).toBe(true);
  const direct = await sofia.from('calls').select().eq('tenant_id', TENANT_B);
  expect(direct.data).toHaveLength(0);                          and updates must fail too
});
```

If this test is red, nothing merges

Not a hotfix, not a small UI change, nothing. A tenant leak is the one bug in this product that cannot be apologised for after the fact, because by then a client has read another client's customer phone numbers.

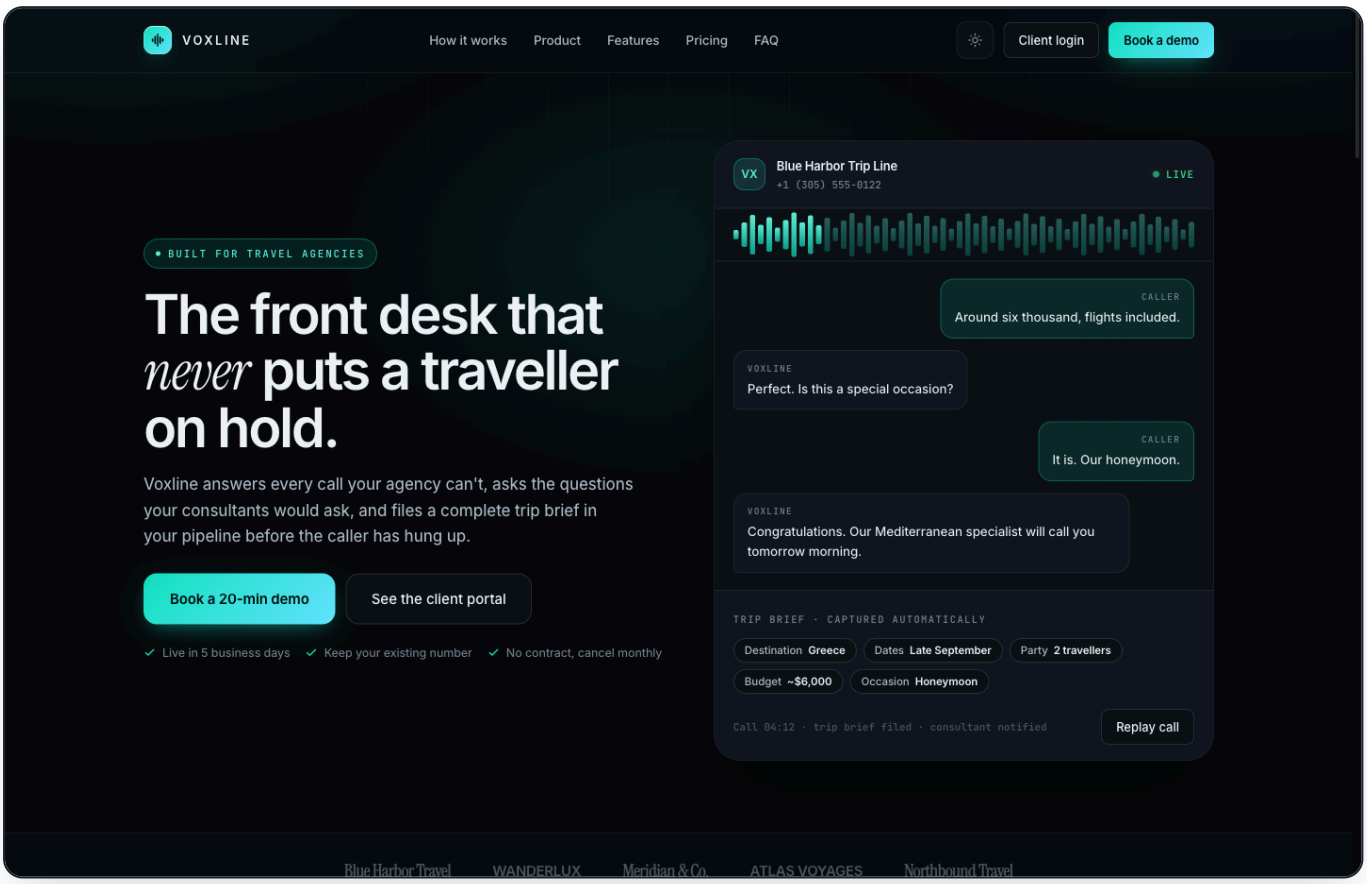
Where the service role key is allowed

CONTEXT	SERVICE ROLE	WHY
Webhook handlers `/api/webhooks/*`	yes	Retell is not a logged-in user, so there is no session to scope. The tenant is resolved from the agent ID
Admin server code `/admin`	yes	Platform admins legitimately cross tenants. Gate it on a platform-admin role check first
Portal server components	no	Use the user's session so RLS applies. This is the whole point
Anything a browser downloads	never	One leaked service key exposes every tenant at once

PART 05 · THE SCREENS

What you are building, on screen

Every picture in this part is a screenshot of `Voxline-UI-Prototype.html` running in a browser, at 1440 pixels wide, using the same seed data you will have in your database. This is not a mood board. It is the target. When your build looks like the picture and behaves like the notes underneath it, that screen is done.



01 · MARKETING PAGE, DARK PROTOTYPE · SITE VIEW · DAY 9, DEPLOYED AS IS

1 The hero runs a live call
Transcript bubbles stream in and the trip-brief chips fill as the agent captures each field. It is scripted, not real, and it is the single most persuasive thing on the page. Do not simplify it away.

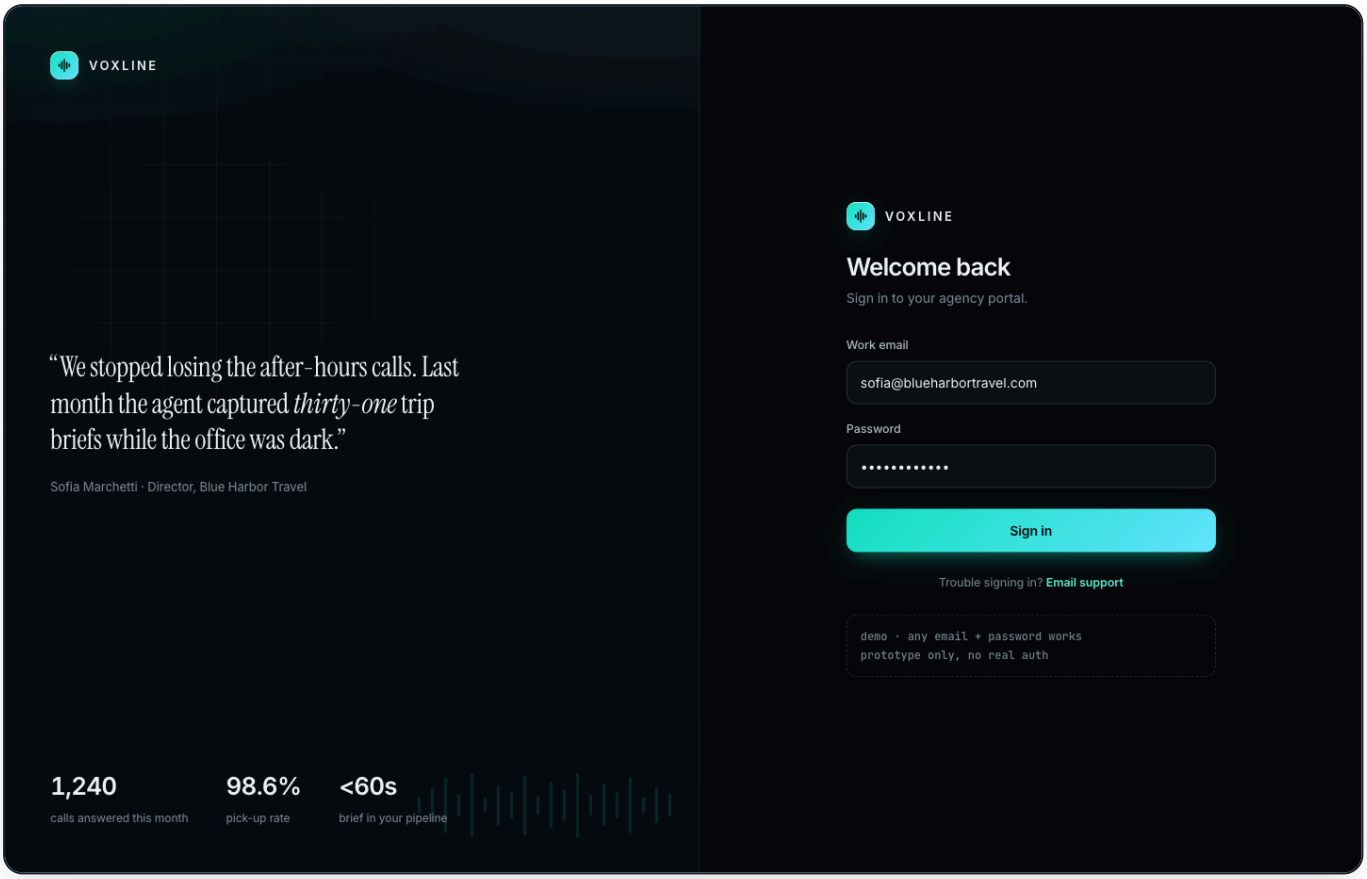
2 One conversion path
Every button on the page goes to the same place: book a demo. No secondary funnels, no newsletter, no chat widget.

3 Ship it exactly as it is
On day 9 this becomes the landing page with no rebuild. Strip the floating Site / Login / Portal pill, add favicon, meta description and OG image, point "Client login" at `/app`.

4 Density is deliberate
Sections are 72px apart and every section heading is a two-column split. If you find yourself adding whitespace, you are undoing a decision.

Login

The first screen a new client ever sees, and the one that decides whether they believe the price. Two fields, and a brand panel doing the persuading.



02 · LOGIN, DARK

DUE END OF DAY 2

1 Split screen, brand left

Ambient gradient, a masked grid, a client quote in serif, three proof metrics along the bottom and the waveform watermark. None of it is decoration you can skip: the right half is only two inputs.

2 Supabase Auth, email and password

Password reset by email works from day one. Magic link is optional and not worth a day.

3 Where they land

One tenant goes straight to its dashboard. Several tenants go to the last used one, remembered in `profiles`. Never show a tenant chooser as an interstitial.

4 Four states

Empty fields, wrong credentials, network failure, and the in-flight state on the button. The red error block above the form is already designed in the prototype.

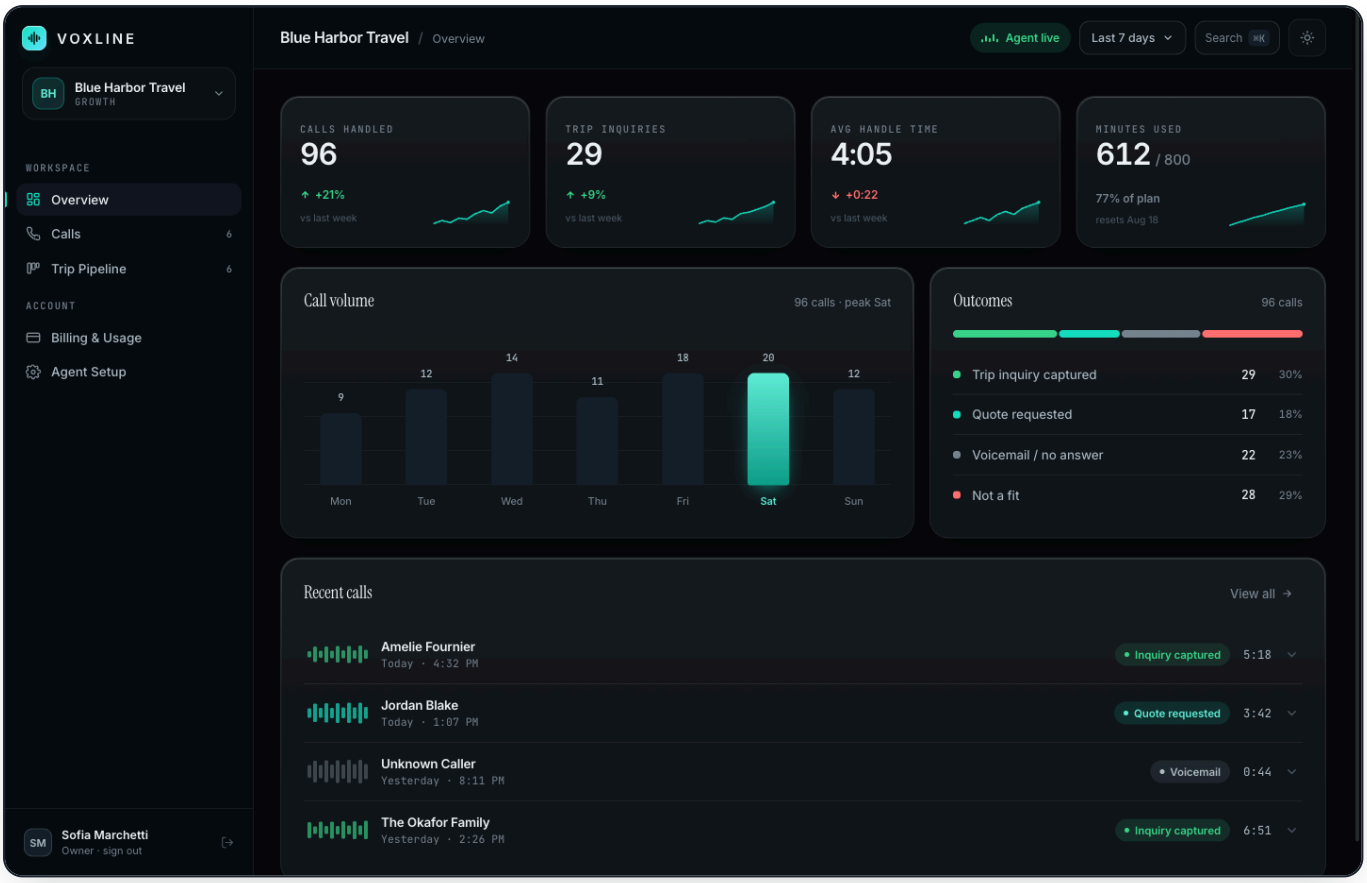
5 Delete the demo hint

The dashed "any email works" box is a prototype aid. It must not exist in the real build.

6 Done when

A seeded owner logs in, a wrong password shows the error, and a refresh keeps them signed in.

Overview, the screen they open every morning



03 · OVERVIEW, DARK

DUE END OF DAY 3

1 Sidebar and switcher

Agency switcher at the top, five nav items with live counts, user chip with sign out at the bottom. The active item carries a two-pixel accent marker on its left edge.

2 Topbar

Agency name, current tab, the live status pill built from the waveform motif, the date range control, the search affordance with its keyboard hint, and the theme toggle.

3 Four KPI cards

Mono uppercase label, a large tabular figure, a delta against the previous seven days, and a gradient sparkline of the last ten points. Minutes used also shows the plan fraction.

4 Call volume

Calls per day, value labelled above every bar, peak day filled with the brand gradient and its weekday label in accent.

5 Outcomes

A segmented proportion bar, then a list with counts and percentages. Green captured, teal quoted, grey voicemail, red not a fit.

6 Recent calls

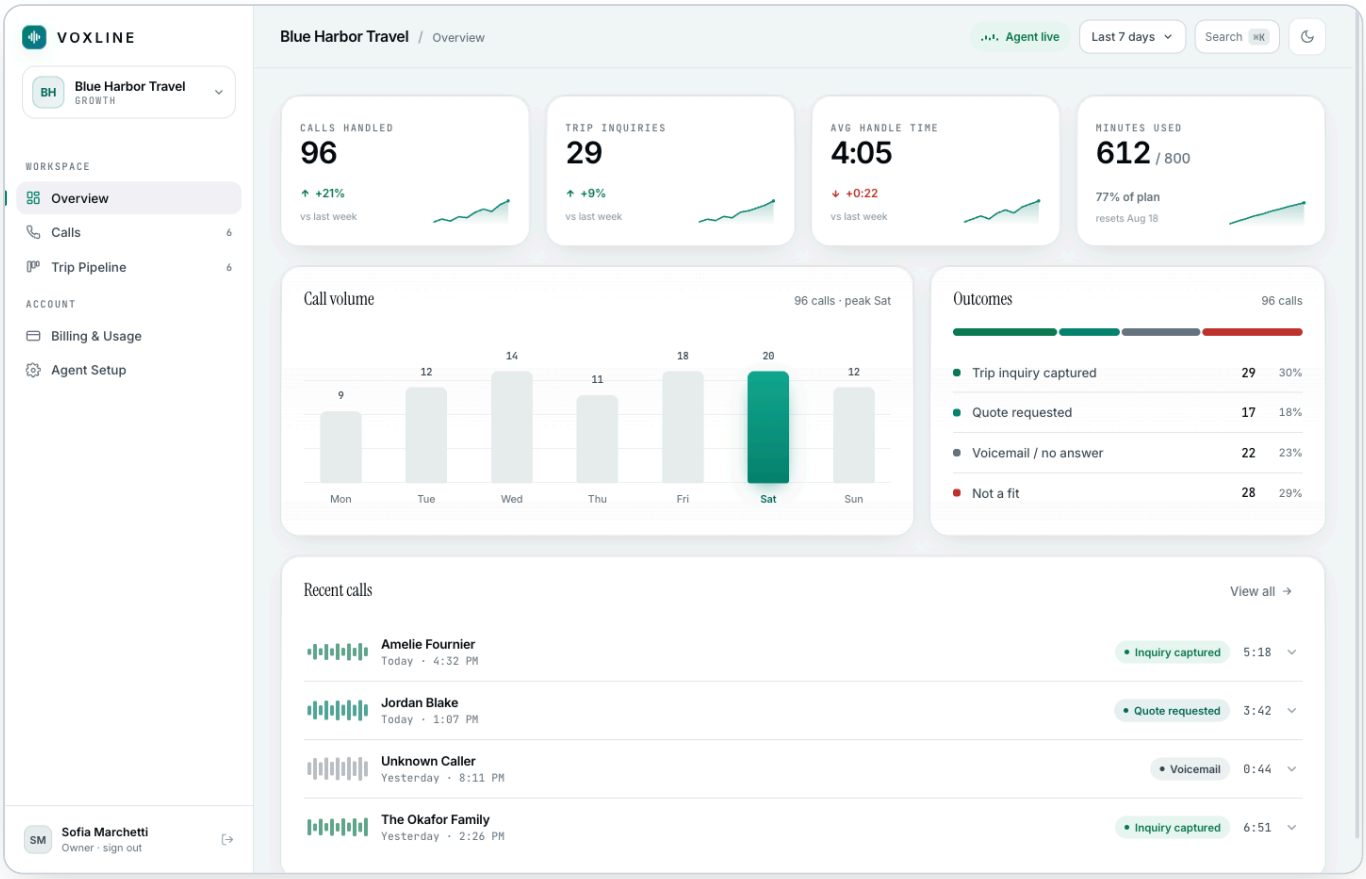
The four newest rows, using the identical component as the Calls tab. Do not build a second row component.

Done when

Every figure reconciles with a manual SQL count against the seed data, and a tenant with zero calls renders clean zeros rather than `NaN` or an empty chart frame.

The same screen in light mode

Light is not a courtesy, it is half the product. Some agency owners work in a bright office and will never see the dark theme. Every screen you build gets checked in both before you call it done.



04 · OVERVIEW, LIGHT

SAME COMPONENT TREE, DIFFERENT TOKENS

1 Nothing moved

Identical layout, identical spacing. Only the token values changed. If your light mode needs a different component, your tokens are wrong.

2 The accent changes value

Electric teal on near-black becomes a deeper teal on white, because `#14E0C0` on white fails contrast. That swap lives in the token layer, never in a component.

3 Shadows carry the depth

Dark leans on borders and a one-pixel inner highlight. Light leans on soft shadows. Both are already defined as tokens.

4 Default is system

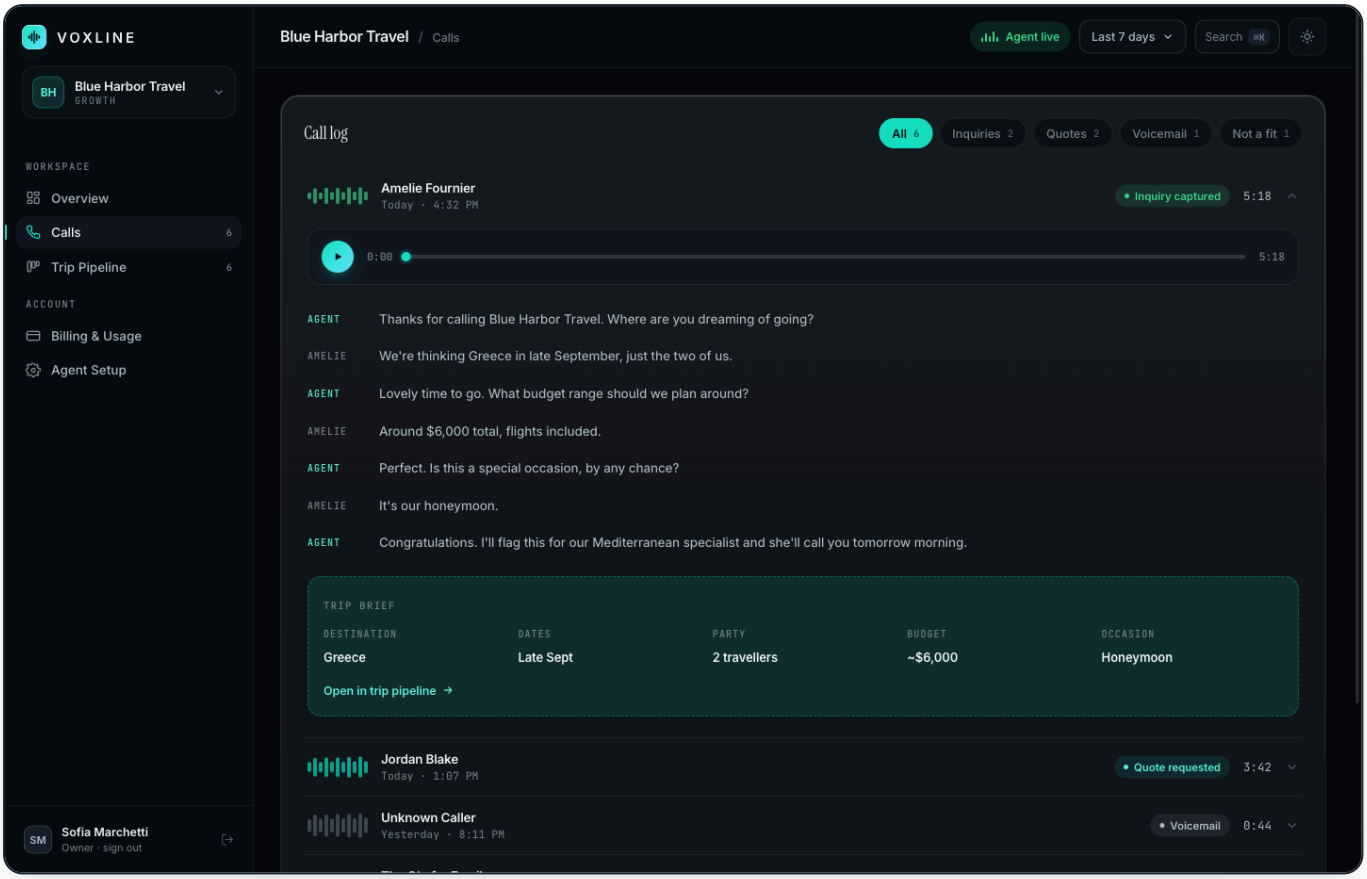
First visit follows the operating system. After that the user's choice is remembered in `profiles.theme_pref`, with a localStorage fallback so it does not flash on load.

The cheapest bug to prevent

Hard-coding one colour in one component. It will look fine in the theme you were working in and broken in the other, and you will not notice for a week. Every colour comes from a token, with no exceptions.

Calls, where the client checks our work

An agency owner opens a call, hears the agent handle their customer well, and stops worrying. This screen has to be flawless, and it is two days of work.



05 · CALLS, ONE ROW EXPANDED DUE END OF DAY 5

1 Filter chips with live counts
All, Inquiries, Quotes, Voicemail, Not a fit, mapped to the `outcome` column. The active chip is a solid accent fill with dark ink.

2 The collapsed row
Mini waveform tinted by outcome, caller name or "Unknown Caller", mono timestamp, outcome badge, duration, caret. Only one row open at a time.

3 Real audio playback
Working play, pause, scrub and elapsed time against a short-lived signed URL from private storage. The prototype fakes the progress; you must not.

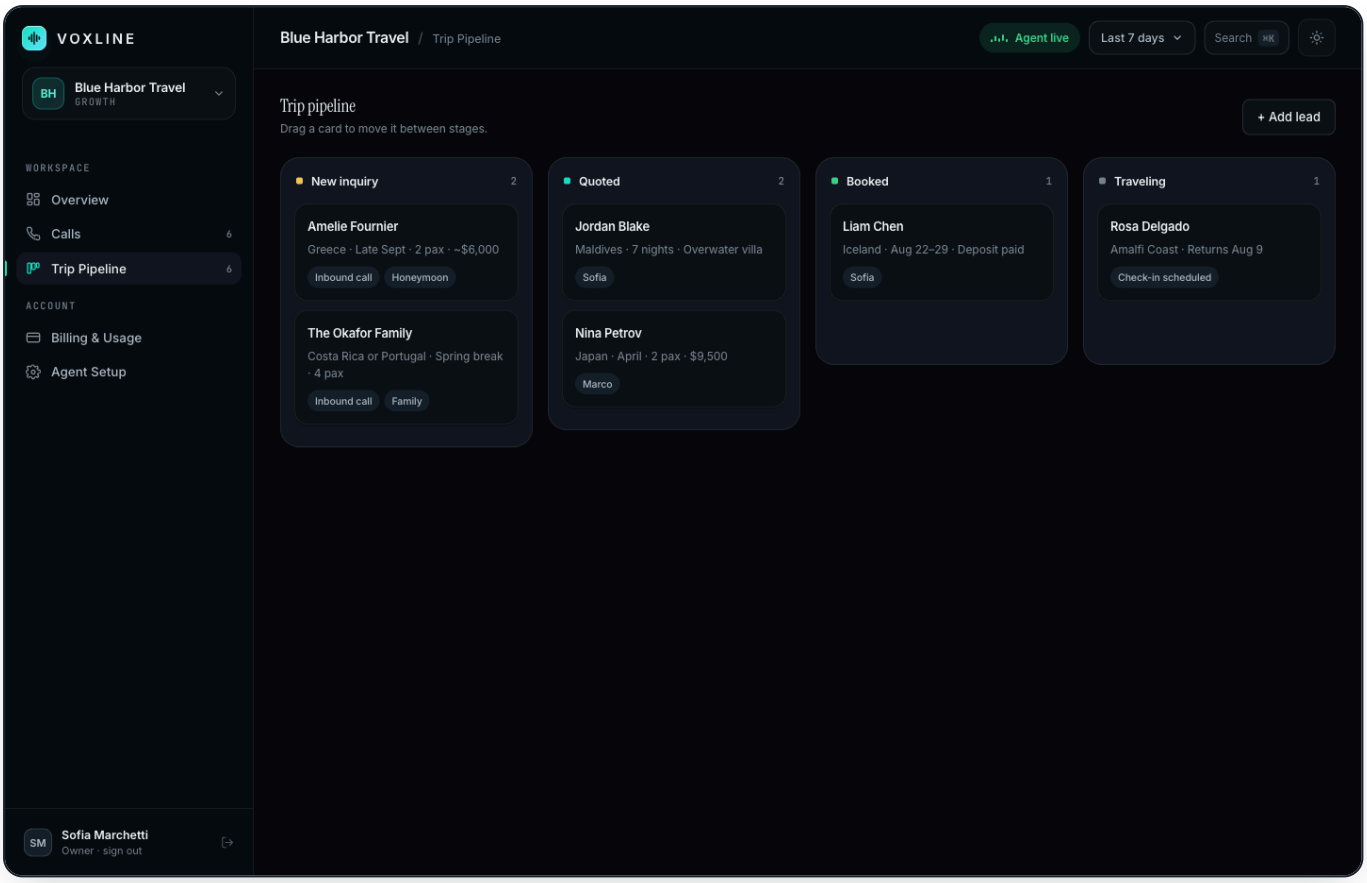
4 Speaker-labelled transcript
Agent turns labelled in accent, caller turns in muted, read straight from `transcript jsonb`. Mono labels, 64px column, comfortable line height.

5 The trip brief
Destination, dates, party, budget, occasion from `analysis jsonb`, in a dashed accent panel, with a link through to the matching lead in the pipeline.

6 The awkward cases
A voicemail has no trip brief, so render none rather than an empty panel. A call with no recording still expands, with the player disabled and labelled.

Done when
A recording actually plays end to end, and the same storage path pasted into a logged-out browser returns an error rather than the audio.

Trip pipeline, the reason they log in twice



06 · TRIP PIPELINE

DUE END OF DAY 6

1 Nobody types these cards

The webhook creates a lead whenever a call's outcome is `inquiry_captured` or `quote_requested`. The summary line is built from the analysis fields.

2 Four stages, coloured dots

New inquiry in warning amber, Quoted in accent, Booked in positive green, Traveling in muted. Each header carries a live count.

3 Drag and drop, persisted

Moving a card writes `stage` and `position`. Update optimistically, roll back on failure, toast either way.

4 Empty columns are designed

A dashed drop zone with the waveform loader and the words "Drop a trip here". Never an empty rectangle.

5 Card anatomy

Name, one summary line, and tags for source and consultant. Hover lifts the card two pixels. Grab cursor while dragging.

6 Responsive

Four columns at desktop, two under 1100px, one under 640px. Drag still has to work with a touch pointer.

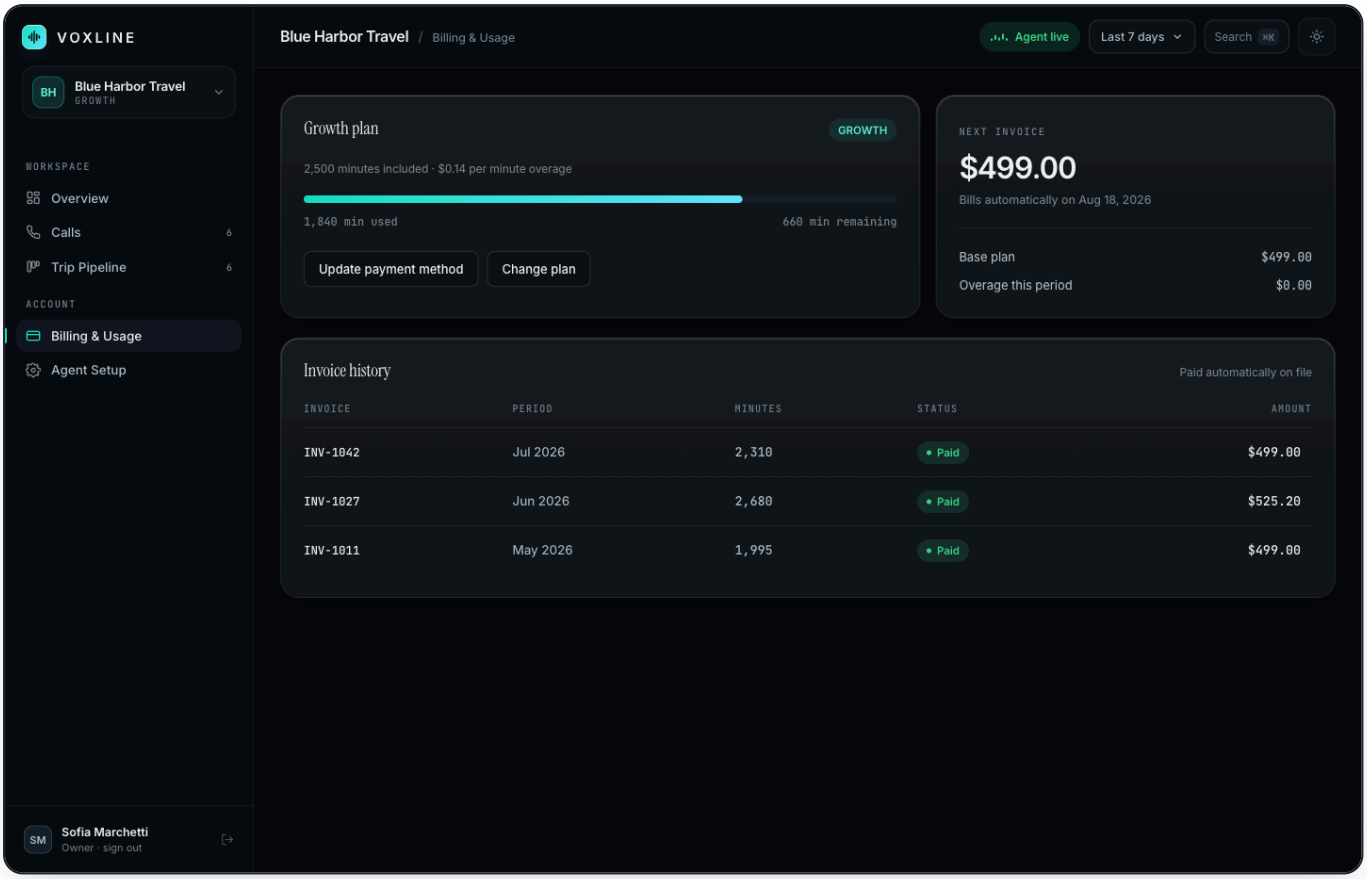
If drag and drop eats more than half a day

Ship a stage dropdown on each card instead and tell Khush the same afternoon. This is the first planned cut on the list, precisely so it does not become a two-day hole. What you must never ship is cards that cannot move at all.

Billing and usage

Phase 2, pulled forward

This screen is why the two-week version is worth shipping. Without it we invoice by hand and chase payment by email, which is not a productised service.



07 · BILLING AND USAGE DUE END OF DAY 8

1 Usage bar, live
Minutes used against included minutes from `usage_periods`, written by the webhook on every call. The bar animates its width once on entry.

2 Next invoice, real
Amount and auto-bill date pulled from the Stripe upcoming invoice, split into base plan and overage so the number is never mysterious.

3 Update payment method
A redirect into the Stripe hosted customer portal. We never build a card form and we never touch card data.

4 Invoice history
Synced from Stripe webhooks into `invoices`, with the status badge and a link to the Stripe-hosted PDF.

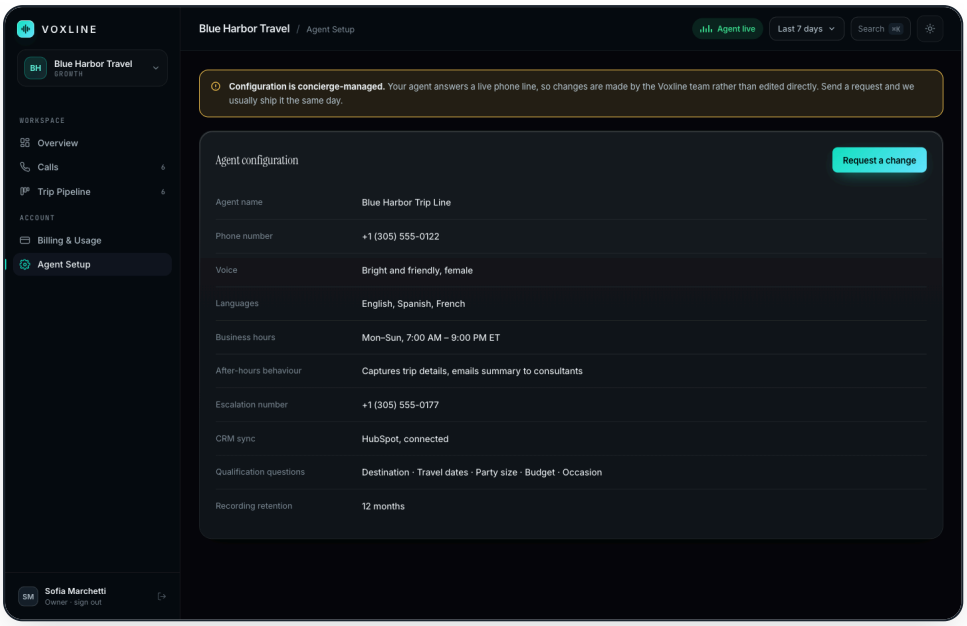
5 Change plan is a conversation
The button opens the same change-request modal as Agent setup. Self-serve upgrades are a later decision, not a two-week one.

6 Payment failure
`invoice.payment_failed` emails the owner and shows a banner in the portal. It never pauses the agent. Pausing is a manual decision Khush makes.

Split of work

The Stripe subscription, the metered usage reporting and the webhook handlers are ticket S-3, built on the senior track. You build this screen against the tables those handlers fill, plus the redirect into the customer portal.

Agent setup, and the phone

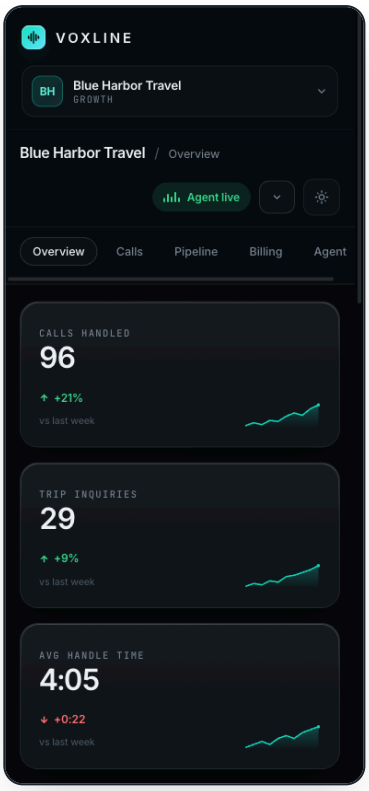


08 · AGENT SETUP, READ ONLY DUE END OF DAY 7

1. **Concierge by design**
   The amber notice explains why this is read only. Say it in the interface rather than letting a client discover a missing edit button and assume it is broken.

2. **A key and value list**
   Straight from `voice_agents`: name, number, voice, languages, hours, after-hours behaviour, escalation number, CRM status, qualification questions, retention.

3. **Request a change**
   The modal writes a `change_requests` row, emails the team through Resend, and toasts "we reply within one business day". That promise is now a real commitment, so the email has to actually arrive.



09 · 390PX

Responsive rules

Sidebar becomes the horizontal tab strip under 1000px. KPI grid goes to two columns under 1100px, one under 560px. Kanban goes two, then one. Tables scroll inside their own container, never the page. Test at 1440, 1024 and 390 before any screen is done.

Every screen, at a glance

SCREEN	DUE	PRIMARY TABLE	THE ONE THING IT MUST DO
Login	Day 2	auth, memberships	Get them in, and look worth paying for
Overview	Day 3	calls	Prove the line is producing
Calls	Day 5	calls	Play the recording, show the brief
Trip pipeline	Day 6	leads	Move a card and have it stay moved
Agent setup	Day 7	voice_agents	Transparency without edit rights
Billing	Day 8	usage_periods, invoices	No surprises on the invoice
Admin	Day 9	tenants, voice_agents	Onboard an agency with no SQL

PART 06 · THE DESIGN SYSTEM

Colour: dark is the default

Map these into the Tailwind theme on day one. After that, a raw hex value anywhere in a component is a bug.

Dark, the default surface

bg #06090C	surface #0C1116	surface-2 #111820	surface-3 #18212B	text #EDF2F5	muted #77848F
accent #14E0C0	accent-2 #63E6FF	positive #3BD68B	warning #F5C451	negative #FF6E6E	accent-ink #03120F

Light, the full override

bg #F4F6F8	surface #FFFFFF	surface-2 #F1F4F6	surface-3 #E7ECEF	text #070D12	muted #66737D
accent #07836F	accent-2 #0C7490	positive #0B7A50	warning #9A6206	negative #C0342E	accent-ink #FFFFFF

Accent discipline is the entire design

The accent covers about five percent of any screen: primary buttons, the active nav marker, the peak bar, sparklines, the live indicator, focus rings. Nothing else. If a screen starts to look teal, take accent away rather than adding more. This single rule is the difference between the prototype and a template.

Type, space, and material

Three typefaces, three jobs

Inter

Everything by default. Body at 15px, line height 1.6. Weights 450 body, 500 controls, 600 headings.

Instrument Serif

A change of voice, not decoration. One or two words in a marketing headline, card and panel titles in the portal, empty states, modals.

JetBrains Mono

Labels, timestamps, phone numbers, invoice IDs, table headers, plan codes. 10 to 11px, uppercase, tracking 0.12em.

Scale and tracking, tighter as it gets bigger

STEP	SIZE	TRACKING	USED FOR
Display	clamp 40 to 58px	-0.042em	Marketing hero only
H2	clamp 27 to 38px	-0.036em	Section heads
Panel title	24px serif	-0.012em	Portal page titles
Card title	19px serif	-0.008em	Card headers in the portal
Body	15px	-0.005em	Everything readable
Micro label	10.5px mono	+0.13em	KPI labels, table headers
KPI figure	32px tabular	-0.04em	Numbers, always tabular-nums

Space and geometry

- Everything sits on an 8pt grid. Card padding 24px, page padding 28px, gaps 16px.
- Radii: 8, 11, 14, 20, 28, and full. Cards 20px, buttons 11px, pills full.
- Marketing sections are 72px apart, not 120px.
- Section heads are two-column splits. A heading that leaves half a row empty is a bug.

Material, what makes it feel expensive

- **Glass fill.** A faint top-to-bottom translucent gradient over every card.
- **Inner highlight.** A one-pixel inset top edge so the card catches light.
- **Gradient hairline.** A border bright at the top, gone by 40 percent down.
- **Spotlight.** Hero and login only, a soft radial following the pointer, under ten percent opacity.
- **Grain.** A fixed low-opacity noise overlay across the page.

Motion

140 to 220ms, easing `cubic-bezier(.2,.7,.3,1)`. Hover lifts one pixel, press settles to 0.985 scale. KPI figures count up once on mount, the usage bar animates on entry. Nothing loops except the live indicator. Honour `prefers-reduced-motion` everywhere.

The waveform motif, and the components

Voxline is a voice company, so the identity is a waveform used as a system rather than a logo parked in a corner. It appears in six places, and nowhere else.

USE	WHAT IT IS	WHERE IT APPEARS
Mark	Five bars in a rounded square, brand gradient, inner white stroke	Nav, footer, login, sidebar, favicon
Wordmark	`VOXLINE, 600, tracking 0.19em`	Always beside the mark. The tracking does not change with size
Divider	Masked repeating waveform, faded at both ends	Section transitions, above the closing call to action
Watermark	The waveform ghosted at 13 percent opacity	Two wide feature cards, the login panel. Maximum one per viewport
Live indicator	Five bars animating on a staggered delay	The portal status pill, the "answering now" card
Empty-state art	The same animation at 35 percent opacity	Filtered call list with no results, empty pipeline column

> **Motif rules**
> Never stretched, never rotated, never in a colour other than the accent, never more than twice on one screen. If it starts to feel like decoration, remove one.

Components to build, with these names

Brand
Logo · WaveRule
WaveWatermark
WaveLoader · Spotlight

Primitives
Button · Badge · Card
Input · Modal · Toast
ThemeToggle · Eyebrow

Data display
KpiCard · Sparkline
BarChart · OutcomeBars
DataTable · UsageBar

Portal
SidebarNav · Topbar
TenantSwitcher · StatusPill
UserChip · CommandPalette

Calls
CallRow · Transcript
AudioPlayer · TripBrief
FilterChips

Pipeline
KanbanColumn
LeadCard

Content
KeyValueList · Notice
EmptyState · LoginCard

Feedback
Skeleton · Delta
RangePicker

Interaction rules that are part of the design

- Focus is always visible: a 2px accent outline at 2px offset. Never removed to make something look neater.
- Every asynchronous action produces a toast. Every destructive action asks first.
- Every list has an empty state written as a real sentence.
- Every number renders correctly at zero, charts included.
- Cmd or Ctrl and K opens the command palette, Cmd or Ctrl and J toggles the theme, Escape closes any overlay.
- Nothing scrolls horizontally except a wide table inside its own scroll container.

PART 07 · THE TEN DAY PLAN

Two weeks, and what has to be true

Monday 24 August to Friday 4 September. Ten working days, no extension. A schedule this tight only works because four things were decided before you started, and because we cut a specific list of features rather than hoping everything fits.

1

You start from a working repo

The day-zero kit is already in GitHub: Next.js with TypeScript strict, Tailwind with our tokens mapped, Supabase clients, Sentry, CI, and Vercel preview deploys. You are not spending a day on setup.

2

The risky backend is not yours

Schema and RLS (S-1), Retell ingestion (S-2) and Stripe billing (S-3) are built by Khush or Vineet on a parallel track. Those three are where an unfamiliar developer loses a week, and the schedule has no week to lose.

3

Reviews come back same day

Pull requests get a response within four working hours. If you are waiting on review, you are blocked, and at this pace a day of blocked time is ten percent of the project.

4

You design nothing

Every screen already exists in the prototype and is photographed in part 5. Your job is to reproduce it with real data, not to make visual decisions. That alone is worth three days.

Two tracks, running in parallel

Track A · Adnan · 10 days

Design system, portal shell, all six client screens, admin console, landing page, hardening. Everything a client sees.

Track B · Khush or Vineet · about 4 days spread across the two weeks

S-1, days 1 to 2. Schema, RLS policies, seed data, the isolation test in CI.
S-2, days 2 to 4. Retell webhook ingestion, idempotency, lead creation, minute counting, the nightly reconciliation job.
S-3, days 6 to 7. Stripe subscription, metered usage reporting, invoice and payment-failure webhooks.

Read this once, then get on with it

Ten days is aggressive for this scope. It is achievable because of the four conditions above, and it stops being achievable the moment one of them slips. If the kit is late, if a review sits for a day, or if the senior track does not deliver S-1 by end of day 2, the honest response is to cut from the list on page 24, not to work a weekend and hope. Tell Khush the same afternoon you see it coming.

Week one 24 to 28 August

Day 1 Mon

Read, set up, build the primitives

Morning: read this book, click every screen of the prototype in both themes, get the kit running locally. Afternoon: Button, Badge, Card, Input, Modal, Toast, ThemeToggle, plus the brand components, on a dev-only kitchen-sink route.

`ships: kitchen sink in both themes` `track B: S-1 starts`

Day 2 Tue

Auth, tenancy, and the portal shell

Login screen against Supabase Auth, protected routes, last-used tenant, the agency switcher, sidebar, topbar, the mobile tab strip and the command palette. Every tab renders an empty panel by the end of the day.

`ships: login + shell` `track B: S-1 done, S-2 starts`

Day 3 Wed

Overview

Four KPI cards with sparklines and period-over-period deltas, the call volume chart with its peak highlight, the outcome breakdown, and the recent calls list. All computed from real rows in the seed data.

`ships: Overview` `zero state must be clean`

Day 4 Thu

Calls, part one

Paginated list, filter chips with live counts, the row component, expand and collapse, the speaker-labelled transcript, and the per-filter empty states.

`ships: call list + transcript` `track B: S-2 done, real calls flowing`

Day 5 Fri

Calls, part two, and the first demo

Real audio playback from signed URLs, the trip-brief block and its link through to the lead. Then the week-one demo: Khush logs in as both agencies and clicks everything.

`ships: playback + trip brief` `demo 1 · 4pm`

The week one bar

By Friday evening a real call placed through the sandbox agent should appear in the portal with a playable recording, a readable transcript and a complete trip brief, in both themes, on a phone. If that is true, the second week is comfortable. If it is not, we cut on Monday morning rather than at the end.

Week two 31 August to 4 September

Day 6 Mon

Trip pipeline, and the date range

The four-stage kanban, drag and drop persisting stage and position, optimistic updates with rollback, designed empty columns. Then wire the 7, 30 and 90 day selector in the topbar so every Overview figure recomputes.

`ships: pipeline` `Phase 2: date ranges`

Day 7 Tue

Agent setup, change requests, and instant alerts

The read-only configuration view, the concierge notice, the change-request modal writing to the database and emailing the team. Then the new-inquiry alert: when a qualified call lands, the agency gets an email within a minute with the trip brief in it.

`ships: agent setup` `Phase 2: new-inquiry email` `track B: S-3 starts`

Day 8 Wed

Billing and usage, live

Usage bar from real usage periods, next invoice from Stripe, base and overage split, the redirect into the Stripe customer portal, invoice history with hosted PDF links, and the payment-failure banner.

`ships: billing` `Phase 2: real Stripe` `track B: S-3 done`

Day 9 Thu

Admin console, landing page, daily digest

Create a tenant, attach a Retell agent and number, set a plan, pause or resume, work the change-request queue, view as tenant read-only. Deploy the prototype marketing page with the preview pill stripped. Ship the 7am digest email.

`ships: admin + landing` `Phase 2: daily digest`

Day 10 Fri

Hardening, and go live

Playwright suite green, isolation test proven, Sentry receiving from production, README written, a bug bash against both tenants, then the pilot cutover and the final demo.

`ships: production` `demo 2 · 3pm` `buffer day if week one slipped`

Day 10 is the buffer, and it is the only one

If you reach Friday with everything done, spend it on the things that make it feel finished: loading skeletons, error copy, keyboard paths, the empty states nobody looks at. Do not start anything new.

What is in, what was cut, and in what order to cut more

shipping in ten days

In scope

- Login, tenancy, agency switching
- Overview, Calls, Trip pipeline, Agent setup
- Real call ingestion, recordings, transcripts
- **Billing live on Stripe**, usage, invoices, customer portal
- **Date range selector**, 7 / 30 / 90 days
- **New-inquiry email** within a minute of the call
- **Daily digest email** at 7am
- Admin console, landing page, monitoring

Bold items are Phase 2 work pulled forward, because without them this is a dashboard rather than a productised service.

cut to make the date

Out of scope

- CRM sync to HubSpot or Salesforce
- Member seats and invites, owner login only
- Manual lead creation, editing and consultant assignment
- Search across calls by name or phone
- Self-serve plan upgrades, these stay a conversation
- White label: tenant logo, accent, subdomain
- CSV export, cross-tenant admin analytics
- A rebuilt marketing site with a demo funnel

None of these block a pilot. All of them are cheaper to build once we have watched two real agencies use the product for a month.

If the schedule slips, cut in this order

ORDER	WHAT GOES	WHAT WE LOSE, AND WHAT REPLACES IT
1	Drag and drop on the pipeline	A stage dropdown on each card. Same capability, worse feel, saves most of a day
2	The daily digest email	The instant new-inquiry alert already covers the important case
3	30 and 90 day ranges	Ship the fixed 7 day view the prototype already shows
4	Admin console polish	Khush onboards the first agencies with a SQL snippet and we build the console in week three
5	The command palette	A power-user nicety. Nobody has ever cancelled over it

What can never be cut

Row-level security and the isolation test. Webhook signature verification. Private recordings behind signed URLs. Both themes working. If cutting these is ever on the table, we move the date instead, because shipping without them is worse than shipping late.

PART 08 · HOW WE WORK

Cadence, and what is expected of you

every working day

Standup note, 9:30am

Three lines in the project channel, no ceremony:

```
Yesterday: call list + filters done
Today: audio playback + trip brief
Blocked: none
```

On a ten-day project this is the only early warning system we have.

daily, 5pm, fifteen minutes

End of day check-in

You screen share what shipped today on the preview URL. Not the code, not a description. The thing.

Two of these are full demos: Friday day 5 and Friday day 10. The other eight are five minutes of clicking and a decision if something is behind.

The forty-minute rule

On a normal project it is an hour. Here it is forty minutes. Stuck that long on the same problem, post in the channel: what you are trying to do, what you tried, the exact error text. Nobody is judged for asking. People are judged for going quiet.

Git, and how work moves

1 · BRANCH

feat/day4-calls-list

One day's work per branch, named for what it is

→

2 · COMMITS

Small and present tense

"add filter chips to call list", not "wip" or "fixes"

→

3 · PULL REQUEST

Open it by 4pm

CI green, preview live, description says what to click

→

4 · MERGE

Same day, always

Nothing sleeps in review. A branch open overnight is a branch that rots

Definition of done, all eight or it is not done

1. Matches the screenshot in part 5, in both light and dark
2. Works at 1440px, 1024px and 390px
3. Loading, empty and error states exist and are not blank panels
4. Typecheck and lint clean, no new `any`
5. The Playwright path for that screen passes
6. The cross-tenant isolation test still passes
7. The preview deployment works and you clicked through it yourself
8. The pull request says how to verify it

What scope means here

Do

- Build what this book shows, at the quality the screenshots show
- Make your own calls inside a day's work: structure, naming, tests
- Say the same afternoon if a day's work will not land that day
- Cut from page 24 rather than working a weekend

Do not

- Add anything not in this book, however small it seems
- Quietly drop a state, a theme or a breakpoint to save an hour
- Add a dependency without asking what it replaces
- Refactor yesterday's screen while today's is unfinished

PART 09 · RULES THAT DO NOT BEND

Seven security rules

These are not guidelines and they are not up for a trade against the date. Every one exists because breaking it ends a client relationship rather than causing a bug report.

1. **Row-level security is on for every tenant table, and the isolation test runs in CI**
   If that test is red, nothing merges. Not a hotfix, not a copy change, nothing.

2. **The service-role key never reaches a browser**
   Server code in admin routes and webhook handlers only. Never behind a `NEXT_PUBLIC_` prefix, never imported into a client component.

3. **Every webhook verifies its signature before doing any work**
   Retell and Stripe both. An unverified webhook endpoint is an open write API to your database.

4. **Recordings live in a private bucket, served through short-lived signed URLs**
   Call recordings contain travellers' names, phone numbers and travel plans. A public bucket URL is a permanent leak.

5. **No real personal data in commits, screenshots, fixtures or channels**
   Use the seeded demo tenants. Blue Harbor and Wanderlux are fictional for exactly this reason.

6. **No new dependency without asking**
   Say what it does and what it replaces. Every package is a supply-chain risk and something we maintain forever.

7. **Your local environment never points at production**
   Development Supabase project only. Check the URL before you run anything that writes or resets.

Tell Khush the same hour, not the next morning

Any suspected cross-tenant leak, however small · any key that may have been exposed · anything you changed that touches a live client phone number · a webhook handler that has been failing and may have lost calls · a day's work that will not land · a requirement you do not understand well enough to build correctly.

None of these get you in trouble to report. All of them get you in trouble to hide.

PART 10 · DEMO DAY

The six checks on Friday 4 September

At 3pm on day 10, Khush sits down and does these six things without touching a terminal. Six passes and we put two real agencies on it the following Monday.

1. **Onboard a brand new agency from the admin console**
   Create the tenant, attach its Retell agent and phone number, set the plan. No SQL, no developer involved.

2. **Call that number and have a normal conversation**
   The agent greets by the agency name and works through the qualification questions.

3. **Watch it land within 60 seconds, and watch the email arrive**
   The call in the portal with transcript, playable recording and trip brief, and the new-inquiry alert in the inbox.

4. **Drag the new lead from New inquiry to Quoted, then refresh**
   It is still in Quoted. Check the billing tab: the minutes from that call are on the usage bar.

5. **Log in as the other agency and find nothing from the first**
   Not in the UI, and not through the API either.

6. **Do all of it on a phone, in dark mode**
   And have it look like a product worth $499 a month.

Handover checklist, same afternoon

- [ ] Playwright suite green in CI: login, expand a call, move a stage, cross-tenant denial
- [ ] Sentry receiving errors from the production deployment
- [ ] README covering setup, environment variables, migrations, seeding and deploy
- [ ] Every environment variable documented in `.env.example` with a comment
- [ ] The nightly reconciliation job proven by deleting a call row and watching it return
- [ ] Stripe in live mode, with one real invoice generated against a test agency
- [ ] A ten-minute recorded walkthrough of the codebase
- [ ] Every known issue written down and handed over, rather than left to be discovered

One last thing

Two real travel agencies are going to run their business on this from Monday. When you are deciding whether a rough edge is worth another twenty minutes, picture a consultant opening this at 7am with a coffee, working out who to call back first. Build for that person and the rest follows.

VOXLINE

BUILD HANDBOOK V2.0 · AUGUST 2026

Questions go to Khush, any time, in the project channel